
机器人集成小组项目 1 实验报告

基于 YOLOv8 和 Jetson Orin NX 的
桌面目标检测与 ROS2 发布

数据集

1000 张图片

正式测试

21/22, 95.45%

完整循环速度

平均 16.77 FPS

推理阈值

conf = 0.70

提交人 QYY

项目仓库 <https://github.com/ccx8kg5qpd-debug/robotics-group-project-1>

报告模板 个人设计 LaTeX 模板

报告日期 2026-08-29

目录

1	实验名称	2
2	实验目的	2
3	实验设备与软件环境	2
3.1	训练环境	2
3.2	部署环境	2
4	数据集构建	3
4.1	自采数据	3
4.2	外部公开数据	3
4.3	最终划分与类别统计	3
5	数据质量检查	3
6	模型训练	4
7	模型评估	4
8	Jetson 部署与实时程序	5
9	ROS2 集成	5
10	正式测试与 FPS	6
11	典型错误与改进	7
12	演示视频	7
13	版本管理与个人仓库	7
14	实验问题与解决过程	8
15	实验总结	8

1. 实验名称

机器人集成小组项目 1：基于 YOLOv8 和 Jetson Orin NX 的桌面目标检测与 ROS2 发布。

2. 实验目的

本实验面向桌面场景中的 bottle 和 mouse 两类物体，完成数据整理与标注、YOLO 目标检测模型训练、Jetson Orin NX GPU 部署、实时检测结果显示及 ROS2 发布。实时界面需要显示 bounding box、class、confidence 和完整循环 FPS；正式测试不少于 20 个实例，正确识别率不低于 80%，Jetson 实际运行速度不低于 5 FPS。

3. 实验设备与软件环境

3.1 训练环境

- Mac，使用 Apple MPS 训练；
- YOLOv8n，输入尺寸 640，训练 100 epochs；
- 训练产物记录的 Ultralytics 版本为 8.4.128。

3.2 部署环境

表 1: Jetson 部署环境

项目	配置
硬件	NVIDIA Jetson Orin NX, USB 摄像头
操作系统	Ubuntu 22.04.5 LTS
JetPack / L4T	JetPack 6.2.1 / L4T R36.4.4
Python	3.10.12
ROS2	Humble
CUDA	约 12.6
Ultralytics	Jetson 部署记录为 8.4.132
PyTorch	与 JetPack 6 / CUDA 12.6 匹配的 ARM64 版本

部署过程验证了 ROS2 OK、YOLO OK 以及 `torch.cuda.is_available() == True`。摄像头由 `cv2.VideoCapture(0)` 打开。

4. 数据集构建

4.1 自采数据

项目最初使用 iPhone 拍摄 30 张真实桌面图片，包括 bottle、mouse 和两类同时出现的场景。原始格式主要为 HEIC，之后转换为 JPEG。早期曾使用 YOLOv8x COCO 预训练模型辅助检查自采图片中的目标框，并人工查看预览结果。该过程只服务于早期自采数据整理。

4.2 外部公开数据

为扩充训练规模，加入 970 张 COCO 2017 图片。外部标注直接来自 COCO 官方 instance annotations，经类别映射后转换为 YOLO 格式，并非由早期自动标注程序生成。外部图片仅用于课程教学和科研实验，每张图片的来源 URL、许可名称、许可 URL、COCO ID 与 SHA-256 均保存在 dataset/source_info/source_manifest.csv。

4.3 最终划分与类别统计

表 2: 最终数据划分

划分	图片/标注	自采	外部	bottle 框	mouse 框
train	800	20	780	2358	696
val	100	5	95	227	82
test	100	5	95	297	95
合计	1000	30	970	2882	873

类别映射为 0=bottle、1=mouse。标注采用 YOLO 归一化格式：class x_center y_center width height。

5. 数据质量检查

数据质量检查遍历 1000 张图片和 1000 个标注文件，检查图片/标注文件名对应、空标注、类别 ID、字段数量、坐标解析、归一化范围、正宽高、边框越界、图片完整性以及跨划分 SHA-256 完全重复。

核验结果为：缺标注 0、孤立标注 0、空标注 0、非法类别 0、非法或越界坐标 0、损坏图片 0、跨划分完全重复图片 0。实际总框数为 bottle 2882、mouse 873。历史统计中的 bottle 2855、mouse 846 是仅针对 970 张外部数据的统计，不能替代最终 1000 张数据的真实数量。

6. 模型训练

训练命令如下：

```
yolo detect train \
  model=yolov8n.pt \
  data=final_dataset/data.yaml \
  epochs=100 \
  imgsz=640 \
  device=mps
```

训练完成 100 epochs, `results.csv` 最后一行记录累计时间 14421.7 秒, 约 4.006 小时。最终部署模型使用 `weights/best.pt`, 大小 6,244,266 bytes。

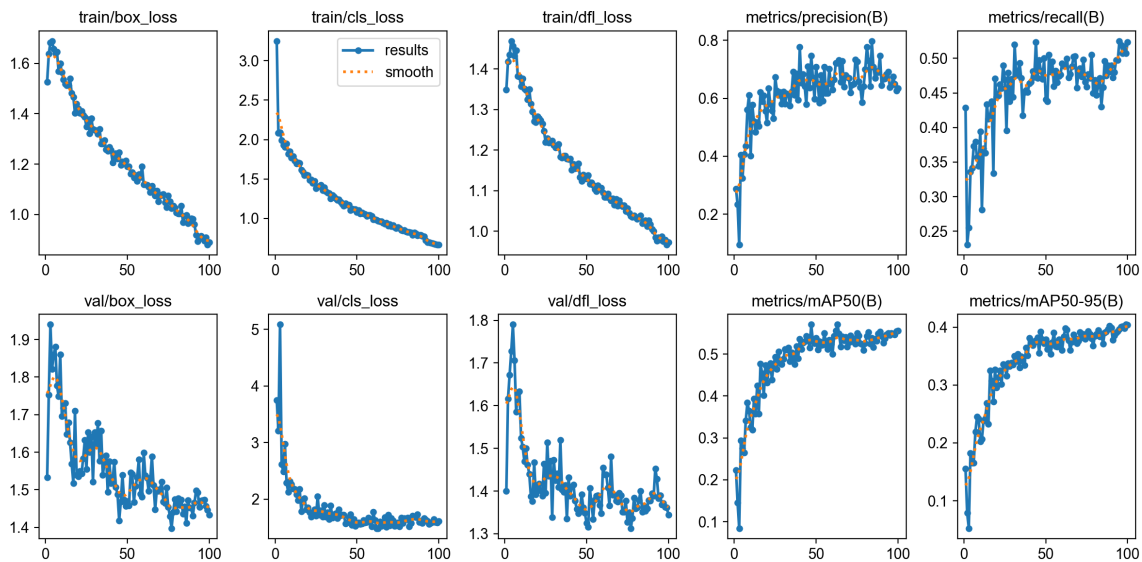


图 1: YOLOv8n 训练曲线

7. 模型评估

训练记录中 mAP50-95 最高的第 99 epoch 与第 100 epoch 汇总指标如下。

表 3: 训练记录汇总指标

来源	P	R	mAP50	mAP50-95
第 99 epoch	0.625	0.508	0.554	0.404
第 100 epoch	0.634	0.523	0.556	0.404

使用保存的 `best.pt` 在 Mac CPU、Ultralytics 8.4.128 上对 100 张 val 图片重新评估, 得到：

表 4: *best.pt* 复评结果

类别	images	instances	P	R	mAP50	mAP50-95
all	100	309	0.676	0.488	0.555	0.405
bottle	63	227	0.592	0.379	0.418	0.276
mouse	62	82	0.760	0.598	0.691	0.533

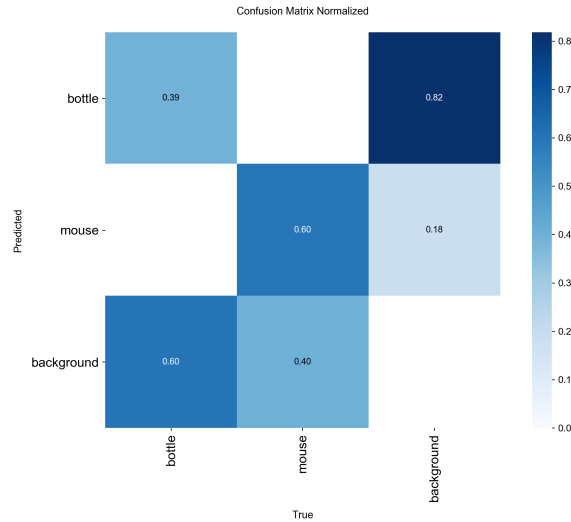


图 2: 归一化混淆矩阵

8. Jetson 部署与实时程序

最终权重部署于 `/home/nvidia/qyy_object_detection`。为保证 GPU 可用，安装与 JetPack 6 / CUDA 12.6 匹配的 ARM64 PyTorch，并处理 `libcudss.so.0` 动态库依赖，将 `/usr/lib/aarch64-linux-gnu/libcudss/12` 加入 `LD_LIBRARY_PATH`。

`detect_realtime.py` 和 `ros2_detect.py` 均加载当前目录下的 `best.pt`，使用 `camera 0`、`device=0` 和 `conf=0.70`。程序显示 `bbox`、`class` 与 `confidence`，并用最近 30 帧完整循环耗时计算平滑 FPS。按 `s` 保存正确案例、按 `e` 保存错误案例、按 `q` 退出。

9. ROS2 集成

`ros2_detect.py` 定义 ROS2 node `qyy_yolo_detector`，创建 `/qyy/detections` publisher，消息类型为 `std_msgs/msg/String`。每帧发布 JSON 字符串，包含 FPS、类别、置信度与 `bbox`。

```
{
  "fps": 18.2,
  "detections": [
```

```
{
  "class": "bottle",
  "confidence": 0.93,
  "bbox": [100, 100, 300, 400]
}
```

上述数字仅用于说明消息格式，不是实验测试结果。项目使用 `ros2 topic list` 与 `ros2 topic echo /qyy/detections` 验证 topic 能持续输出实时 JSON 检测结果。

10. 正式测试与 FPS

`correct_cases` 和 `error_cases` 中的保存帧作为正式测试实例。共记录 22 个实例，其中 21 个正确、1 个错误，正确率为 95.45%，高于作业要求的 80%。唯一错误是横放 bottle 漏检；同一帧中的 mouse 以 0.96 置信度正确识别，没有发生错误类别。

表 5: 正式测试汇总

测试实例	正确	错误	正确率	作业要求	结论
22	21	1	95.45%	≥20 且 ≥80%	达标

22 张保存帧显示的完整循环 FPS 为 16.6-16.9，算术平均约 16.77 FPS，最低值、最高值和平均值均高于 5 FPS。该数值包含摄像头采集、推理、绘制与显示，不是由单独 inference time 反算。



图 3: bottle 与 mouse 双目标正确案例，FPS 16.9

11. 典型错误与改进



图 4: 真实错误案例: 横放 bottle 漏检, mouse 正确识别

初期使用较低 confidence threshold 时, 键盘区域曾被误识别为 mouse, 出现过约 0.26 和约 0.63 的错误置信度。真实 mouse 常见约 0.96/0.98, 真实 bottle 常见约 0.95/0.96。将推理阈值提高到 0.70 后, 该类已知误检受到抑制。此处改动是推理阈值调整, 并非重新训练模型。键盘误检图片未保留, 因此只记录调试现象, 不附模拟截图。

12. 演示视频

最终演示视频原件为 IMG_9829.MOV, 原始画面 3840x2160、约 119.95 FPS、时长约 104.08 秒。课程提交包中包含 1920x1080、30 FPS、约 104.07 秒的 final_demo.mp4 画面版。

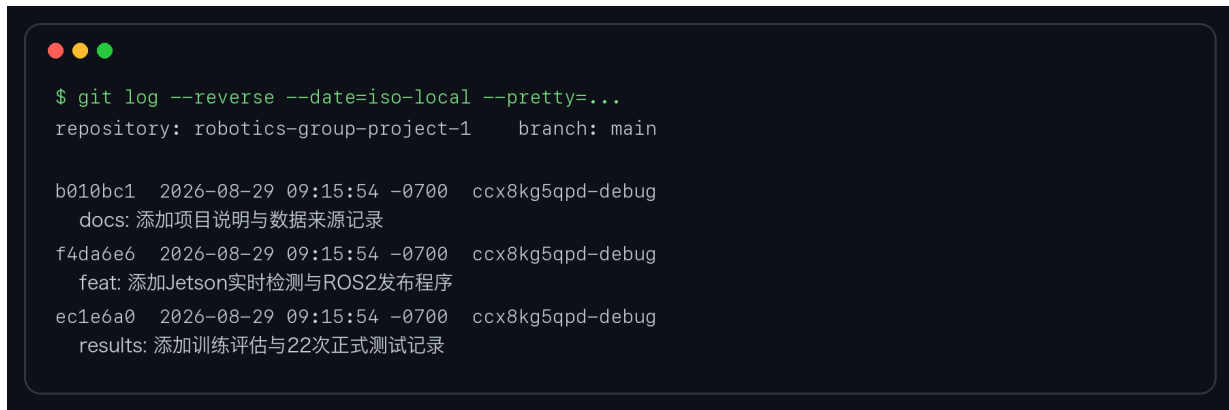
分段抽帧确认视频展示了 Jetson 硬件、USB 摄像头、bottle、mouse、双目标检测、bbox/class/confidence/FPS 叠加, 以及另一个终端持续输出 /qyy/detections JSON 消息。提交副本首帧和 100 秒位置均可正常解码。

13. 版本管理与个人仓库

项目最终材料建立了独立 Git 仓库, 并按内容阶段提交, 未沿用参考项目的 Git 历史。个人仓库地址为:

<https://github.com/ccx8kg5qpd-debug/robotics-group-project-1>

提交分为数据来源与说明、Jetson/ROS2 程序与模型、训练和正式测试结果、LaTeX 报告等阶段。图 5 为报告编制时的真实本地 commit 记录; 远程推送后可在上述 GitHub 仓库继续查看完整记录。



```
$ git log --reverse --date=iso-local --pretty=...
repository: robotics-group-project-1    branch: main

b010bc1 2026-08-29 09:15:54 -0700 ccx8kg5qpd-debug
docs: 添加项目说明与数据来源记录

f4da6e6 2026-08-29 09:15:54 -0700 ccx8kg5qpd-debug
feat: 添加Jetson实时检测与ROS2发布程序

ec1e6a0 2026-08-29 09:15:54 -0700 ccx8kg5qpd-debug
results: 添加训练评估与22次正式测试记录
```

图 5: *Git* 分阶段提交记录

考虑到 GitHub 文件大小限制和外部图片许可, 完整 1000 图数据集、原始演示视频及课程提交 ZIP 不在公共仓库重复发布。仓库保留 `data.yaml`、划分清单、数据质量报告和逐图来源/许可清单, 完整材料随课程提交 ZIP 提交。

14. 实验问题与解决过程

- 数据来源表述: 最终 1000 张中只有 30 张为自采, 970 张来自 COCO 2017;
- Jetson PyTorch/CUDA 兼容: 使用与 JetPack 6 / CUDA 12.6 匹配的 ARM64 PyTorch;
- 动态库问题: 安装 `libcudss0-cuda-12` 并配置 `LD_LIBRARY_PATH`;
- 桌面误检: 将推理置信度阈值调整为 0.70, 抑制已知 keyboard-to-mouse 误检;
- ROS2 发布: 采用 `std_msgs/String` 发送 JSON, 并通过 `topic echo` 验证。

15. 实验总结

项目完成了 bottle 与 mouse 两类桌面目标的数据集整理、YOLOv8n 训练、Jetson Orin NX GPU 实时部署、bbox/class/confidence 显示、完整循环 FPS 测量、正确/错误案例保存及 ROS2 结果发布。数据集规模为 1000 张, `best.pt` 复评 `mAP50-95` 为 0.405。22 个正式测试实例中 21 个正确, 正确率 95.45%; 平均完整循环速度约 16.77 FPS, 满足测试数量、正确率和实时速度要求。最终演示视频、个人 Git 仓库、分阶段 commit 记录以及个人 LaTeX 报告模板均作为课程材料整理。